

## Exercice 5 — Mapping JPA — relation @OneToMany / @ManyToOne

EILMS — Java Enterprise (Step 1 & Step 2)

### Énoncé

Écris l'entité WarehouseEntity liée à WarehouseItemEntity par une relation @OneToMany (un entrepôt a plusieurs items).

### Correction

```
@Entity
@Table(name = "warehouses")
public class WarehouseEntity {

    @Id
    @Column(name = "warehouse_id")
    private String id;

    @OneToMany(mappedBy = "warehouse", cascade = CascadeType.ALL)
    private List<WarehouseItemEntity> items = new ArrayList<>();

    // getters/setters
}

// Dans WarehouseItemEntity, il faut ajouter le côté propriétaire :
@ManyToOne
@JoinColumn(name = "warehouse_id")
private WarehouseEntity warehouse;
```

mappedBy = "warehouse" indique que c'est WarehouseItemEntity qui possède réellement la clé étrangère (via @JoinColumn). Le côté @OneToMany n'est qu'une vue pratique côté Java — il ne crée aucune colonne supplémentaire en base.

cascade = CascadeType.ALL signifie que si tu sauvegardes ou supprimes un WarehouseEntity, Hibernate répercute automatiquement l'opération sur tous ses items associés. Exemple :  
em.remove(warehouse) supprimera aussi tous ses WarehouseItemEntity en base.

**Erreur fréquente** : oublier mappedBy et mettre @JoinColumn des deux côtés — Hibernate créerait alors deux colonnes de clé étrangère différentes, ou une table de jointure inutile.